MODUL PRAKTIKUM

PEMROGRAMAN BERORIENTASI OBJEK

PENYUSUN:

Timotheus Ferdinand Tjondrojo (NIM: 105222019)

Yan Stephen Christian Immanuel Hutagalung (NIM: 105222010)

LABORATORIUM KOMPUTER

PROGRAM STUDI ILMU KOMPUTER

FAKULTAS SAINS DAN ILMU KOMPUTER

UNIVERSITAS PERTAMINA

2026

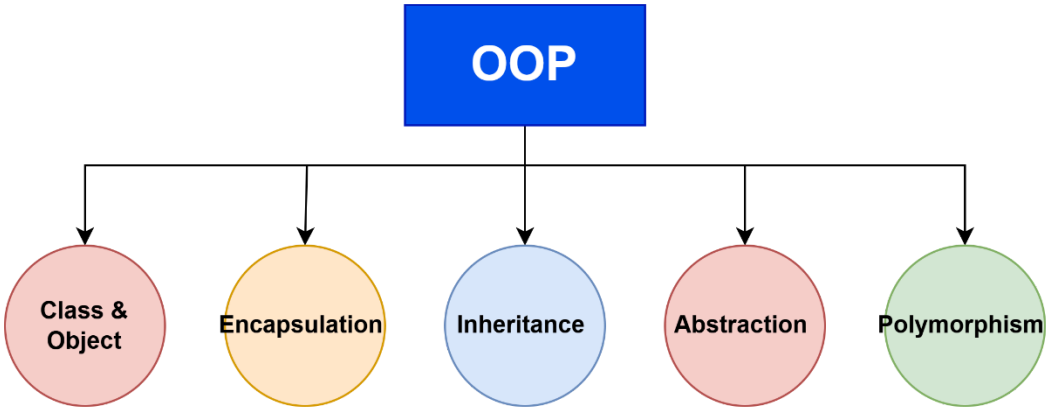
The Core Of OOP

PreTest

Kamu memiliki entitas data seorang mahasiswa yang memiliki nama, umur, dan jurusan studi. Tugasmu adalah memetakan entitas ini ke dalam struktur OOP. Tentukan apa nama class yang paling merepresentasikan entitas tersebut, sebutkan ketiga atribut utamanya beserta tipe data yang paling masuk akal, dan terakhir, tuliskan baris kode Java secara spesifik yang menunjukkan bagaimana cara membuat satu objek nyata dari blueprint class tersebut dengan memberikan nama objek mhsBaru.

Pengenalan OOP

OOP (Object-Oriented Programming) atau Pemrograman Berorientasi Objek adalah paradigma pemrograman yang berfokus pada penggunaan objek untuk merancang dan mengimplementasikan program. OOP membantu dalam membuat kode yang lebih terstruktur, mudah dikelola, dan dapat digunakan kembali.



Konsep Dasar OOP

a) Class

- Dalam Object-Oriented Programming (OOP), kelas (class) adalah kerangka kerja (blueprint) yang digunakan untuk menciptakan objek. Kelas berfungsi seperti cetakan yang mendefinisikan atribut (data/properti) dan metode (fungsi/perilaku) dari objek yang dibuat berdasarkan kelas tersebut.
- Contoh: kelas mahasiswa, kelas kendaraan, kelas hewan, dll.

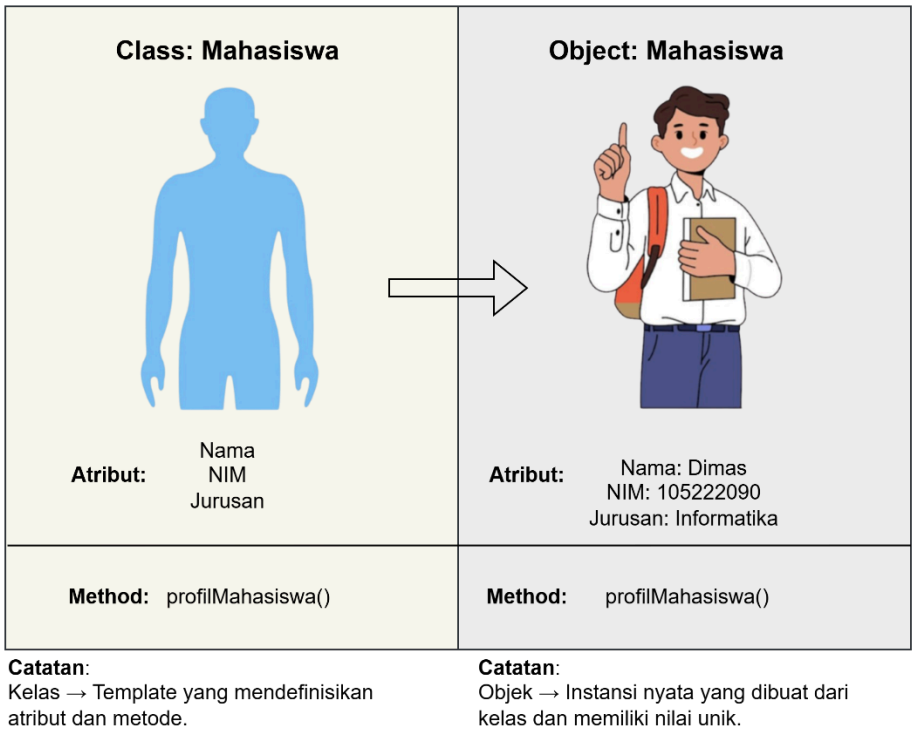
b) Object

Dalam pemrograman berorientasi objek (OOP), objek adalah entitas yang memiliki dua karakteristik utama:

- State (Keadaan/Status) → Merujuk pada atribut atau properti yang dimiliki oleh objek.
- Behavior (Perilaku) → Merujuk pada aksi atau metode yang dapat dilakukan oleh objek.

Contoh objek dalam dunia nyata:

- Buku memiliki state (judul, penulis, jumlah halaman) dan behavior (dibuka, dibaca, ditutup).
- Komputer memiliki state (merk, RAM, prosesor) dan behavior (menyala, memproses data, mati).



c) Encapsulation

- Encapsulation adalah proses mengikat (binding) data dan metode dalam satu unit (kelas), sekaligus membatasi akses langsung ke data tersebut. Hal ini dilakukan agar data dalam suatu objek tetap aman dan tidak bisa diubah sembarangan dari luar kelas.
- Dengan encapsulation, <u>data tetap aman</u> dan hanya bisa dimanipulasi melalui metode yang telah ditentukan, sehingga mencegah perubahan langsung yang bisa merusak logika program.

d) Inheritance

- *Inheritance* (pewarisan) adalah proses di mana suatu kelas (*child class*) mewarisi properti dan metode dari kelas lain (*parent class*), sehingga memungkinkan penggunaan kembali kode tanpa perlu menulis ulang. Dengan *inheritance*, *child class* dapat menggunakan fitur dari *parent class*, menambahkan fitur baru, atau memodifikasi metode yang ada. Hal ini membuat kode lebih modular, terstruktur, dan mudah dikelola.
- Contohnya, jika class Mobil mewarisi class Kendaraan, maka Mobil akan memiliki semua properti dan metode Kendaraan, seperti merk dan jalan(), sambil tetap bisa menambahkan fitur spesifik seperti klakson() atau jumlahPintu.

e) Abstraction

- Dalam OOP, abstraksi adalah teknik yang menyembunyikan detail implementasi suatu sistem dan hanya menampilkan fungsionalitas utama kepada pengguna. Dalam Java, abstraksi dapat dicapai menggunakan kelas abstrak dan interface.
- Contoh nyata dari abstraksi adalah mobil, di mana pengguna hanya melihat dan menggunakan fitur seperti tombol start, persneling, layar tampilan, dan rem, sementara mekanisme internal seperti cara kerja mesin, proses menyalakan mobil, dan perpindahan gigi tersembunyi. Saat pengguna melakukan suatu aksi pada fitur tersebut, sistem akan menjalankan proses internalnya tanpa harus diketahui oleh pengguna.

f) Polymorphism

- Istilah "polymorphism" berarti "banyak bentuk". Dalam pemrograman berorientasi objek, polimorfisme memungkinkan satu entitas memiliki beberapa bentuk dengan nama yang sama. Di Java, polimorfisme dapat diterapkan melalui dua konsep utama, yaitu method overloading dan method overriding.
- Method overloading terjadi dalam satu kelas, di mana terdapat beberapa metode dengan nama yang sama tetapi memiliki perbedaan dalam jumlah atau tipe parameter. Sementara itu, method overriding terjadi melalui mekanisme pewarisan (inheritance), di mana sebuah metode pada kelas induk dapat ditulis ulang dengan cara yang berbeda dalam kelas turunan, tetap menggunakan nama yang sama.

OOP vs Procedural Programming

Pemrograman berorientasi objek (OOP) memiliki beberapa keunggulan dibandingkan dengan pemrograman prosedural:

- OOP lebih cepat dan lebih mudah dieksekusi.
- OOP memberikan struktur yang jelas untuk program.
- OOP membantu menjaga kode Java tetap DRY (*Don't Repeat Yourself*), sehingga kode lebih mudah dipelihara, dimodifikasi, dan diperbaiki.
- OOP memungkinkan pembuatan aplikasi yang dapat digunakan kembali secara penuh dengan lebih sedikit kode dan waktu pengembangan yang lebih singkat.

Tips: Prinsip DRY (*Don't Repeat Yourself*) bertujuan untuk mengurangi pengulangan kode. Artinya kita sebaiknya mengekstrak bagian kode yang sering digunakan dalam aplikasi, menyimpannya di satu tempat, dan menggunakannya kembali daripada menulis ulang kode yang sama.

Aspek	**OOP (Object-Oriented Programming)**	**Pemrograman Prosedural**
Pendekatan	Berbasis objek (menggunakan kelas dan objek)	Berbasis prosedur (menggunakan fungsi dan prosedur)
Struktur Program	Terorganisir dalam bentuk kelas dan objek	Terstruktur secara linier dengan serangkaian instruksi
Keunggulan Utama	Reusability, enkapsulasi, dan modularitas	Lebih sederhana dan mudah dipahami untuk program kecil
Fokus Utama	Entitas dunia nyata (objek) dan interaksi antar objek	Urutan eksekusi dan alur logika program
Data & Keamanan	Data terlindungi melalui enkapsulasi (akses terbatas)	Data cenderung terbuka dan bisa diakses dari mana saja
Pemeliharaan & Skalabilitas	Mudah dipelihara dan dikembangkan untuk proyek besar	Bisa sulit dikelola saat program menjadi lebih kompleks
Penggunaan Ulang Kode	Tinggi, karena mendukung inheritance dan polymorphism	Rendah, sering terjadi duplikasi kode
Contoh Bahasa	Java, Python, C++, PHP (OOP)	C, Pascal, Basic, Python (prosedural)

Implementasi Class dan Object

a) **Class**

Membuat kelas:

- Menggunakan keyword **class**
- Nama kelas harus diawali dengan huruf besar (PascalCase)
- Jika terdiri dari lebih dari satu kata, gunakan **camel case** (contoh: MahasiswaBaru)
- Nama kelas harus **deskriptif** agar mudah dipahami.
- Contoh benar: Mahasiswa, Mobil, PenggunaAkun.
- Contoh salah: mahasiswa, mobil123, pengguna_akun.

Contoh:

```
class Mahasiswa {

}
```

Dalam pemrograman berorientasi objek (OOP), sebuah kelas dapat berisi berbagai elemen yang mendukung fungsionalitasnya. Berikut adalah elemen-elemen yang bisa ada dalam kelas:

1. Atribut (Properties/Fields)

   Atribut adalah variabel yang menyimpan data dalam sebuah kelas. Atribut ini menentukan karakteristik dari objek yang dibuat dari kelas tersebut.

   Contoh:

```
class Mahasiswa {
    // Atribut (Properti/Variabel Anggota)
    private String nama;
    public String nim;
    protected int angkatan;
    boolean isLulus = false;
}
```

   Ketentuan atribut:

   - Menggunakan **camelCase** untuk nama atribut.
   - Bisa memiliki **modifier akses** (private, public, protected).
   - Bisa memiliki nilai awal (default value)

2. Konstruktor (Constructor)

Konstruktor adalah metode khusus yang dipanggil saat objek dibuat. Konstruktor digunakan untuk menginisialisasi nilai atribut.
Contoh:

```
class Mahasiswa {
    // Atribut
    private String nama;
    public String nim;
    protected int angkatan;
    boolean isLulus = false; // default value

    // Contstructor
    public Mahasiswa(String nama, String nim, int angkatan) {
        this.nama = nama;
        this.nim = nim;
        this.angkatan = angkatan;
    }
}
```

Ketentuan konstruktor:

- Nama konstruktor harus sama dengan nama kelas.
- Tidak memiliki return type (tidak menggunakan void atau tipe lainnya)
- Bisa ada lebih dari satu konstruktor (overloading)

3. Method / Function
Metode adalah fungsi dalam kelas yang digunakan untuk melakukan aksi atau mengembalikan nilai.
Contoh:

```
class Mahasiswa {
    // Atribut
    private String nama;
    public String nim;
    protected int angkatan;
    boolean isLulus = false; // default value

    // Contstructor
    public Mahasiswa(String nama, String nim, int angkatan) {
        this.nama = nama;
        this.nim = nim;
        this.angkatan = angkatan;
    }

    // Method / Function
    public void belajar() {
        System.out.println(this.nama + " sedang belajar");
```

```
    }

    public String getNama() {
        return this.nama;
    }
}
```

Ketentuan metode:

- Menggunakan camelCase untuk nama metode.
- Bisa memiliki parameter dan return value
- Bisa menggunakan modifier akses (public, private, dll.)

4. Inner Class (Kelas di dalam kelas)
   Java mendukung kelas dalam kelas lain (inner class) untuk modularitas.
   Contoh:

```
public class Luar {

    class Dalam {
        void tampilkan() {
            System.out.println("Ini adalah kelas dalam.");
        }
    }
}
```

b) Object

Untuk membuat objek dalam Java, kita menggunakan kata kunci **new** untuk mengalokasikan memori untuk objek baru.
Sintaks dasar:

```
NamaKelas namaObjek = new NamaKelas();
```

Contoh:
Class Mahasiswa: (berperan sebagai blueprint untuk membuat objek)

```
public class Mahasiswa {
    String nama;
    String nim;
    int angkatan;
}
```

Membuat objek dari class Mahasiswa:

```
public class Main {
    public static void main(String[] args) throws Exception {
        Mahasiswa mhs1 = new Mahasiswa();
        mhs1.nama = "Budi";
        mhs1.nim = "12345";
        mhs1.angkatan = 2023;
    }
}
```

- Mahasiswa adalah **kelas**.
- mhs1 adalah **objek** dari kelas Mahasiswa

Mengakses / mencetak objek:

```
public class Main {
    public static void main(String[] args) throws Exception {
        Mahasiswa mhs1 = new Mahasiswa();
        mhs1.nama = "Budi";
        mhs1.nim = "12345";
        mhs1.angkatan = 2023;

        // Mencetak data mahasiswa dari objek mhs1
        System.out.println("Nama: " + mhs1.nama);
        System.out.println("NIM: " + mhs1.nim);
        System.out.println("Angkatan: " + mhs1.angkatan);
    }
}
```

Output:

```
Nama: Budi
NIM: 12345
Angkatan: 2023
```

Kita juga bisa mengubah data dalam objek:

```
public class Main {
    public static void main(String[] args) throws Exception {
        Mahasiswa mhs1 = new Mahasiswa("Budi", "12345", 2023);
```

```
        mhs1.profilMahasiswa();

        // mengubah nama mahasiswa
        mhs1.nama = "Andi";

        mhs1.profilMahasiswa();
    }
}
```

Metode dan Fungsi

a) Metode (Method)

Method adalah fungsi yang dideklarasikan di dalam sebuah kelas dan digunakan untuk menjalankan aksi atau perilaku dari objek. Dengan kata lain, method adalah fungsi yang dimiliki oleh objek dalam OOP.

Contoh Method:

```
public class Mahasiswa {
    String nama;
    int umur;

    // Constructor
    public Mahasiswa(String nama, int umur) {
        this.nama = nama;
        this.umur = umur;
    }

    // Method tanpa parameter
    public void tampilkanInfo() {
        System.out.println("Nama: " + nama + ", Umur: " + umur);
    }

    // Method dengan parameter
    public void ubahNama(String namaBaru) {
        this.nama = namaBaru;
    }

    // Method dengan return value
    public int getUmur() {
        return umur;
    }

    public static void main(String[] args) {
        Mahasiswa mhs1 = new Mahasiswa("Budi", 20);
```

```
        mhs1.tampilkanInfo(); // Output: Nama: Budi, Umur: 20

        mhs1.ubahNama("Andi");
        mhs1.tampilkanInfo(); // Output: Nama: Andi, Umur: 20

        System.out.println("Umur: " + mhs1.getUmur()); // Output:
Umur: 20
    }
}
```

- **tampilkanInfo()** → Method tanpa parameter, hanya mencetak informasi mahasiswa.
- **ubahNama(String namaBaru)** → Method dengan **parameter** untuk mengubah nilai atribut nama.
- **getUmur()** → Method dengan **return value** untuk mengembalikan umur mahasiswa.

b) Fungsi (Function)

Fungsi adalah sekumpulan kode yang dapat digunakan berulang kali untuk melakukan suatu tugas tertentu. Fungsi biasanya memiliki nama, parameter (opsional), dan nilai kembalian (return value, opsional).

Contoh fungsi:

```
public class ContohFungsi {
    // Fungsi tanpa parameter dan tanpa return
    public static void sapa() {
        System.out.println("Halo, selamat datang!");
    }

    // Fungsi dengan parameter
    public static void sapaPengguna(String nama) {
        System.out.println("Halo, " + nama + "!");
    }

    // Fungsi dengan parameter dan return value
    public static int tambah(int a, int b) {
        return a + b;
    }

    public static void main(String[] args) {
        sapa(); // Output: Halo, selamat datang!
        sapaPengguna("Budi"); // Output: Halo, Budi!
        System.out.println(tambah(5, 3)); // Output: 8
    }
```

```
}
```

- Fungsi sapa() hanya mencetak teks.
- Fungsi sapaPengguna(String nama) menerima **parameter** untuk menyesuaikan output.
- Fungsi tambah(int a, int b) menerima dua angka dan mengembalikan hasil penjumlahan

c) Static Method VS Non-static Method

- Non-static Method: harus dipanggil melalui **objek** karena terkait langsung dengan objek tertentu.

Contoh:

```
public class Mobil {
    String merk;

    public Mobil(String merk) {
        this.merk = merk;
    }

    public void infoMobil() {
        System.out.println("Mobil ini bermerk: " + merk);
    }

    public static void main(String[] args) {
        Mobil mbl1 = new Mobil("Toyota");
        mbl1.infoMobil(); // Output: Mobil ini bermerk:
Toyota
    }
}
```

Method **infoMobil()** hanya bisa dipanggil melalui objek (mbl1).

- Static Method: Static method tidak perlu objek untuk dipanggil, karena milik **kelas**, bukan objek.

Contoh:

```
public class Kalkulator {
```

```
    public static int tambah(int a, int b) {
        return a + b;
    }

    public static void main(String[] args) {
        System.out.println(Kalkulator.tambah(10, 5)); //
Output: 15
    }
}
```

tambah() bisa langsung dipanggil tanpa membuat objek (Kalkulator.tambah(10, 5)).

d) Method Overloading
Adalah method dengan **nama yang sama tetapi parameter berbeda**.

Contoh:

```
public class Hitung {
    public int tambah(int a, int b) {
        return a + b;
    }

    public int tambah(int a, int b, int c) {
        return a + b + c;
    }

    public static void main(String[] args) {
        Hitung h = new Hitung();
        System.out.println(h.tambah(3, 5)); // Output: 8
        System.out.println(h.tambah(3, 5, 2)); // Output: 10
    }
}
```

Java akan memilih method sesuai dengan jumlah parameter yang diberikan.

Konstruktor

Konstruktor adalah method khusus dalam sebuah kelas yang digunakan untuk menginisialisasi objek saat objek tersebut dibuat. Konstruktor dipanggil secara otomatis ketika objek baru dibuat.

Karateristik konstruktor:

- Nama konstruktor harus sama dengan nama kelasnya.
- Tidak memiliki tipe data pengembalian (return type), bahkan void pun tidak boleh.
- Dipanggil secara otomatis saat objek dibuat.
- Digunakan untuk menginisialisasi atribut objek.

a) Jenis-jenis konstruktor dalam Java

Dalam Java, terdapat tiga jenis konstruktor utama:

1. Konstruktor default (tanpa parameter)

   Konstruktor ini otomatis dibuat oleh Java jika tidak ada konstruktor yang dideklarasikan dalam kelas.

   Contoh:

```
public class Mahasiswa {
    String nama;
    int umur;

    // Konstruktor default (tanpa parameter)
    public Mahasiswa() {
        System.out.println("Konstruktor default dipanggil!");
        nama = "Tidak diketahui";
        umur = 0;
    }

    public void tampilkanInfo() {
        System.out.println("Nama: " + nama + ", Umur: " +
umur);
    }

    public static void main(String[] args) {
        Mahasiswa mhs1 = new Mahasiswa(); // Konstruktor
dipanggil otomatis
        mhs1.tampilkanInfo(); // Output: Nama: Tidak diketahui,
Umur: 0
    }
}
```

   - Konstruktor Mahasiswa() dipanggil otomatis saat objek mhs1 dibuat.
   - Atribut nama dan umur diinisialisasi dengan nilai default.

2. Konstruktor Berparameter

   Konstruktor yang menerima **parameter** untuk menginisialisasi objek dengan nilai yang diberikan saat objek dibuat.

Contoh:

```
public class Mahasiswa {
    String nama;
    int umur;

    // Konstruktor dengan parameter
    public Mahasiswa(String nama, int umur) {
        this.nama = nama;
        this.umur = umur;
    }

    public void tampilkanInfo() {
        System.out.println("Nama: " + nama + ", Umur: " +
umur);
    }

    public static void main(String[] args) {
        Mahasiswa mhs1 = new Mahasiswa("Budi", 20);
        mhs1.tampilkanInfo(); // Output: Nama: Budi, Umur: 20

        Mahasiswa mhs2 = new Mahasiswa("Andi", 22);
        mhs2.tampilkanInfo(); // Output: Nama: Andi, Umur: 22
    }
}
```

- Konstruktor menerima parameter nama dan umur untuk menginisialisasi objek.
- **Keyword this** digunakan untuk membedakan antara variabel instance dan parameter.

3. Konstructor Copy (Copy constructor)
   Adalah konstruktor yang digunakan untuk membuat objek baru dengan **menyalin nilai dari objek lain**.

   Contoh:

```
public class Mahasiswa {
    String nama;
    int umur;

    // Konstruktor dengan parameter
    public Mahasiswa(String nama, int umur) {
        this.nama = nama;
        this.umur = umur;
```

```
    }

    // Konstruktor Copy
    public Mahasiswa(Mahasiswa m) {
        this.nama = m.nama;
        this.umur = m.umur;
    }

    public void tampilkanInfo() {
        System.out.println("Nama: " + nama + ", Umur: " +
umur);
    }

    public static void main(String[] args) {
        Mahasiswa mhs1 = new Mahasiswa("Budi", 20);
        Mahasiswa mhs2 = new Mahasiswa(mhs1); // Salin dari
objek mhs1

        mhs1.tampilkanInfo(); // Output: Nama: Budi, Umur: 20
        mhs2.tampilkanInfo(); // Output: Nama: Budi, Umur: 20
    }
}
```

Konstruktor **copy** digunakan untuk menyalin atribut dari objek mhs1 ke objek mhs2.

4. Overloading Konstruktor
   Java mendukung **overloading konstruktor**, yaitu membuat beberapa konstruktor dalam satu kelas tetapi dengan parameter berbeda.

   Contoh:

```
public class Mahasiswa {
    String nama;
    int umur;

    // Konstruktor 1: Default
    public Mahasiswa() {
        this.nama = "Tidak diketahui";
        this.umur = 0;
    }

    // Konstruktor 2: Dengan satu parameter
    public Mahasiswa(String nama) {
        this.nama = nama;
        this.umur = 18; // Default umur
    }
```

```
    // Konstruktor 3: Dengan dua parameter
    public Mahasiswa(String nama, int umur) {
        this.nama = nama;
        this.umur = umur;
    }

    public void tampilkanInfo() {
        System.out.println("Nama: " + nama + ", Umur: " +
umur);
    }

    public static void main(String[] args) {
        Mahasiswa mhs1 = new Mahasiswa();
        Mahasiswa mhs2 = new Mahasiswa("Budi");
        Mahasiswa mhs3 = new Mahasiswa("Andi", 21);

        mhs1.tampilkanInfo(); // Output: Nama: Tidak diketahui,
Umur: 0
        mhs2.tampilkanInfo(); // Output: Nama: Budi, Umur: 18
        mhs3.tampilkanInfo(); // Output: Nama: Andi, Umur: 21
    }
}
```

Java memilih konstruktor berdasarkan parameter yang diberikan saat pembuatan objek.

5. Keyword this() dalam konstruktor
   this() digunakan untuk **memanggil konstruktor lain** dalam kelas yang sama untuk menghindari kode yang duplikatif.

   Contoh:

```
public class Mahasiswa {
    String nama;
    int umur;

    // Konstruktor 1: Default
    public Mahasiswa() {
        this.nama = "Tidak diketahui";
        this.umur = 0;
    }

    // Konstruktor 2: Dengan satu parameter
    public Mahasiswa(String nama) {
        this.nama = nama;
        this.umur = 18; // Default umur
```

```java
    }

    // Konstruktor 3: Dengan dua parameter
    public Mahasiswa(String nama, int umur) {
        this.nama = nama;
        this.umur = umur;
    }

    public void tampilkanInfo() {
        System.out.println("Nama: " + nama + ", Umur: " +
umur);
    }

    public static void main(String[] args) {
        Mahasiswa mhs1 = new Mahasiswa();
        Mahasiswa mhs2 = new Mahasiswa("Budi");
        Mahasiswa mhs3 = new Mahasiswa("Andi", 21);

        mhs1.tampilkanInfo(); // Output: Nama: Tidak diketahui,
Umur: 0
        mhs2.tampilkanInfo(); // Output: Nama: Budi, Umur: 18
        mhs3.tampilkanInfo(); // Output: Nama: Andi, Umur: 21
    }
}
```

this("Tidak diketahui", 0); memanggil konstruktor lain dengan parameter.

b) Perbedaan Konstruktor dan Method

Perbedaan	Konstruktor	Method
Nama	Sama dengan nama kelas	Bisa diberi nama apa saja
Return Type	Tidak memiliki return type	Bisa memiliki return type (void, int, String, dll.)
Pemanggilan	Dipanggil otomatis saat objek dibuat	Dipanggil secara eksplisit oleh objek
Tujuan	Menginisialisasi objek	Melakukan operasi tertentu

Latihan

Soal 1 :

Buat class bernama MesinKopi yang memiliki atribut biji kopi dalam gram, air dalam mililiter, dan susu dalam mililiter. Setelah blueprint tersebut selesai, beralihlah ke dalam class Main dan ciptakan satu objek nyata dari mesin ini dengan memberikan nama mesinLobby sebagai representasi mesin yang ada di ruang tunggu.

Soal 2 :

Pemilik mesin harus bisa mengisi ulang bahan baku agar operasional tetap berjalan, sehingga kamu perlu menambahkan sebuah method void bernama isiUlangBahan ke dalam class MesinKopi. Method ini harus mampu menerima tiga parameter sekaligus yaitu untuk tambahan kopi, air, dan susu, yang kemudian bertugas menambahkan nilai parameter-parameter tersebut ke kapasitas atribut mesin saat ini dan diakhiri dengan mencetak pesan ke layar bahwa bahan baku berhasil diisi ulang.

Soal 3 :

Mesin harus memiliki kecerdasan dasar untuk mengecek apakah bahan baku di tangki cukup untuk membuat satu cangkir minuman Cappuccino yang standar resepnya membutuhkan 15 gram kopi, 50 mililiter air, dan 100 mililiter susu. Wujudkan fitur ini dengan membuat sebuah method bertipe kembalian boolean bernama cekKetersediaanCappuccino yang secara logis akan mengembalikan nilai true jika seluruh bahan mencukupi, dan mutlak mengembalikan nilai false jika ada satu atau lebih bahan yang tidak memenuhi standar resep.

Soal 4 :

Pabrik baru saja memproduksi mesin kopi tersebut sehingga tangki di dalamnya pasti dalam keadaan kosong. Untuk merepresentasikan inisialisasi awal ini di dalam kodemu, buatlah sebuah constructor default tanpa parameter yang secara otomatis mengeset nilai seluruh bahan (kopi, air, dan susu) menjadi nol mutlak pada saat objek mesin diciptakan pertama kali di memori.

Post Test

Sebuah startup fintech bernama "DuitKu" sedang mengembangkan core engine untuk aplikasi dompet digital mereka. Sesuai aturan perusahaan, setiap pelanggan baru yang bergabung wajib mendaftarkan nama lengkap dan nomor ponsel aktifnya sebagai identitas utama, dan tentunya saldo awal mereka dipastikan selalu berada di angka nol mutlak pada sistem. DuitKu melayani dua aktivitas utama. Aktivitas pertama adalah pengisian dana, di mana pengguna dapat menyetor uang untuk menambah saldo mereka beserta pencetakan struk resi; namun, kebijakan minimum setoran adalah Rp 10.000, sehingga sistem wajib memblokir nominal di bawah itu dan mengeluarkan peringatan keras. Aktivitas kedua adalah pelunasan tagihan, di mana sistem harus bisa memverifikasi kecukupan dana sebelum memotong saldo; sistem harus memberikan sinyal konfirmasi keberhasilan jika dana cukup, atau memberikan sinyal kegagalan mutlak tanpa memotong sepeser pun jika dana tidak mencukupi. Berdasarkan proses bisnis tersebut, bangunlah struktur OOP-nya dengan tepat. Buktikan sistem ini baik dengan mensimulasikan seorang pelanggan baru bernama Anton yang mendaftarkan nomor 081234567. Simulasikan Anton menyetor uang Rp 50.000, lalu ia mencoba melunasi tagihan listrik sebesar Rp 60.000. Setelah sistem menolaknya secara otomatis, Anton menyetor lagi Rp 20.000, dan kembali mencoba melunasi tagihan yang sama hingga akhirnya berhasil. Akhiri simulasi dengan menampilkan sisa uang Anton yang paling akurat di sistem.

Bonus : Gunakan input user